

T1: Attention Recurrent Language Model and Machine Translation

CIMAT, Procesamiento de Lenguaje Natural, Ciencias de la Computación

Profesor: Dr. Adrián Pastor López Monroy

Entregar: Martes 6 de Octubre de 2026 antes de las 23:59:59

Instrucciones

Realiza los siguientes puntos en un notebook de Python *lo mejor organizado y claro posible*. Ponga su nombre al notebook (e.g., `adrian_pastor_lopez_monroy.ipynb`) y también en la primera celda del notebook junto con el número de Tarea. Sube al classroom el notebook como un archivo, que deberá haber sido ejecutado en tu máquina y mostrar el resultado en las celdas.

Los modelos deben ser evaluados con los datos adjuntos. Para la parte de Language Modelling, use Train para entrenar y Val para probar. Para la parte de Machine Translation, use las particiones Train, Val y Test proporcionadas como se indica en el ejercicio correspondiente. Implemente su función de perplexidad; puede usar Vibe Coding. **Es TU responsabilidad si no es correcta la implementación tanto de los modelos como de la métrica.**

1 Language Modelling

Puedes usar *Vibe Coding* para programar el Modelo de Lenguaje Recurrente con Atención con PyTorch. Escoja el LLM de su preferencia y haga los siguientes puntos, COMPLEMENTA CON OBSERVACIONES RELEVANTES O COMENTARIOS EN CADA PUNTO (e.g. TODO ES VALIOSO, SABER QUE MODELO USASTE, DIFICULTADES, ALUCINACIONES, ETC.)

1. (20pts) Con los datos de tuits adjuntos Programe un Modelo de Lenguaje Recurrente con lo aprendido del tema del libro de Dan Jurafsky. Este Modelo de Lenguaje debe tener algún mecanismo de Self Attention que usted decida (e.g., *"hops/query vectors"* como en paper 3 o 4, softmax sobre scores de palabras vs palabras, o mapping de QKV tipo transformer). Decida todos los hyperparametros con su intuición y expertise (e.g., arquitectura recurrente tipo LSTM/GRU/Bidirectional/Stacked, número de capas ocultas y neuronas en ellas, tamaño de la secuencia, etc, etc).
2. (0pts) Implemente el Statistical Language Model de la clase de NLP y reporte su perplexidad. <https://www.youtube.com/watch?v=fXWUzJfolmA>
3. (0pts) Implemente el Neural Language Model de la clase de NLP y reporte su perplexidad. <https://www.youtube.com/watch?v=3qBVXGqaXnM> y <https://www.youtube.com/watch?v=0eobbLL0Ty4>
4. (20pts) Derrote en perplexidad (Leer Capítulo de N-Grams de Dan Jurafsky sino sabe que es la perplexidad) al Statistical Language Model y al Neural Language Model con su Recurrent Language Model with Attention; compare también que sucede si le quita la atención a su modelo. Muestre una pequeña Tabla con sus resultados.
5. (10pts) Escoga dos ejemplos de cada modelo para generar texto automáticamente y haga una pequeña demostración ilustrativa a su gusto. De un comentario general de su experiencia en esta Tarea.

2 Machine Translation

Puede usar *Vibe Coding* para programar un modelo de Traducción Automática con PyTorch. Utilice los archivos de Tatoeba adjuntos: `tatoeba_train.tsv`, `tatoeba_val.tsv` y `tatoeba_test.tsv`. Train contiene el 80% de los pares del corpus original, Val el 10% y Test el 10%. Use exclusivamente Train para entrenamiento, Val para validación y Test para la evaluación final. COMPLEMENTA CON OBSERVACIONES RELEVANTES O COMENTARIOS (e.g. LLM utilizado, decisiones tomadas, dificultades, alucinaciones, recursos de cómputo, tiempo de entrenamiento, etc.).

1. (30pts) Programe un modelo traductor **Seq2Seq** inglés-español con una arquitectura recurrente Encoder-Decoder (e.g., RNN/LSTM/GRU) y un mecanismo de Attention basado explícitamente en **Query, Key y Value (Q, K, V)** tipo Transformer. NO use un Transformer completo como modelo de traducción. Identifique claramente en su implementación cómo obtiene Q, K y V y cómo calcula los pesos de atención.
2. (10pts) Realice dos experimentos con exactamente la misma arquitectura y configuración general de entrenamiento:
 - Experimento 1: entrene con X pares de Train.
 - Experimento 2: entrene con $2X$ pares de Train.

Usted decide el valor de X de acuerdo con sus recursos computacionales. Escoja un valor que le permita completar razonablemente también el experimento con $2X$, ya sea utilizando su computadora, Google Colab (Standard o Pro/Plus) o los recursos del cluster de CIMAT a los que tenga acceso. Debe cumplirse $2X \leq |\text{Train}|$. Para hacer justa la comparación, los X ejemplos del primer experimento deberán ser un **subconjunto** de los $2X$ ejemplos del segundo. Use Val para tomar decisiones durante el entrenamiento (e.g., selección de hyperparametros o *early stopping*). **No utilice Test para tomar decisiones sobre el modelo.**

3. (10pts) Evalúe los dos modelos sobre el mismo archivo Test y reporte al menos las métricas **BLEU** y **chrF**. Puede utilizar la librería `sacrebleu`, que ya implementa ambas métricas. Muestre una pequeña Tabla comparando los resultados para X y $2X$. Finalmente, escoja al menos 10 ejemplos de Test y muestre para cada uno: la oración original en inglés, la traducción de referencia, la traducción generada por el modelo entrenado con X ejemplos y la traducción generada por el modelo entrenado con $2X$ ejemplos. Dé un comentario general sobre los resultados y los errores o diferencias que observe.